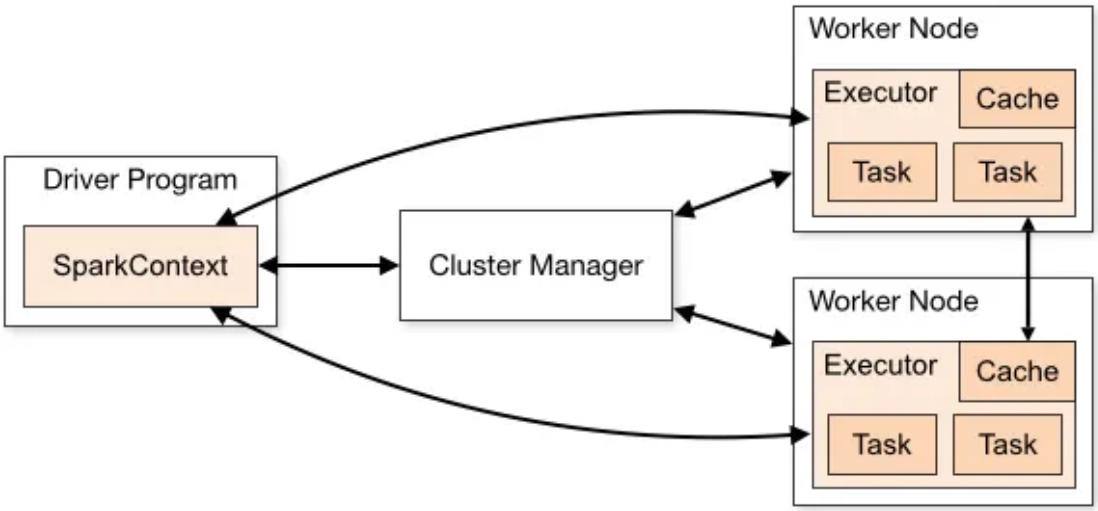
The Apache Spark framework uses a **master-slave architecture** that consists of a driver, which runs as a master node, and many executors that run across as worker nodes in the cluster. Apache Spark can be used for batch processing and real-time processing as well.



Working Of Spark Architecture

When the Driver Program in the Apache Spark architecture executes, it calls the real program of an application and creates a SparkContext. SparkContext contains all of the basic functions. The Spark Driver includes several other components, including a DAG Scheduler, Task Scheduler, Backend Scheduler, and Block Manager, all of which are responsible for translating user-written code into jobs that are actually executed on the cluster. Spark Driver and SparkContext collectively watch over the job execution within the cluster.

Core Components of Spark Architecture

Driver Node

The **Driver Node** is the central controller of a Spark application. It runs the main program and is responsible for managing the entire execution process. It creates the execution plan, divides the job into smaller tasks, and schedules them across worker nodes.

The driver also maintains metadata about jobs and monitors task execution. It communicates with executors to assign work and collect results. In essence, the driver acts as the “brain” of Spark, translating user code into distributed operations.

Worker Node and Executors

Worker nodes are the machines in a cluster where actual computation happens. Each worker node runs one or more **executors**, which are processes responsible for executing tasks and storing intermediate data.

Executors perform operations such as transformations and actions on data. They also cache data in memory to improve performance and send results back to the driver.

This distributed setup allows Spark to process large datasets efficiently by splitting work across multiple machines.

Cluster Manager

The **Cluster Manager** is responsible for resource management in the Spark cluster. It allocates CPU, memory, and other resources to applications and decides where executors should run.

Spark supports multiple cluster managers, including:

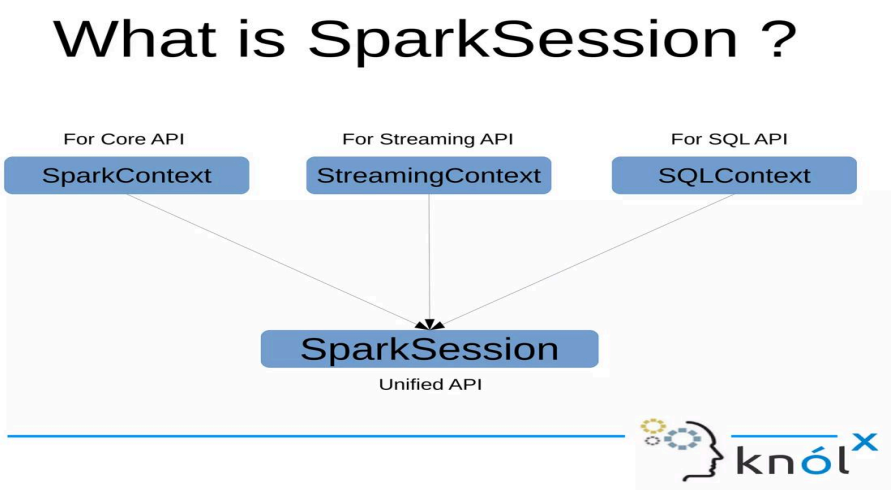
- Standalone (built-in)
- Hadoop YARN
- Apache Mesos
- Kubernetes

The driver communicates with the cluster manager to request resources, and the cluster manager launches executors on worker nodes accordingly.

SparkSession and SparkContext

SparkSession is the unified entry point for working with Spark. It was introduced to simplify the API by combining multiple contexts into one. It allows users to interact with structured data using DataFrames and SQL queries.

Through SparkSession, users can read data, perform transformations, and execute queries. Internally, it manages the SparkContext and provides a higher-level interface.



SparkContext is the core component that establishes communication between the driver and the cluster. It is responsible for resource allocation, task scheduling, and creating distributed datasets (RDDs).

There is typically only one SparkContext per application, and it acts as the foundation of Spark execution. Although SparkSession is now preferred, SparkContext still plays a critical role internally.

Execution Flow in Spark

The execution of a Spark application follows a structured workflow:

1. User submits a Spark application
2. Driver program starts execution
3. SparkSession/SparkContext is created

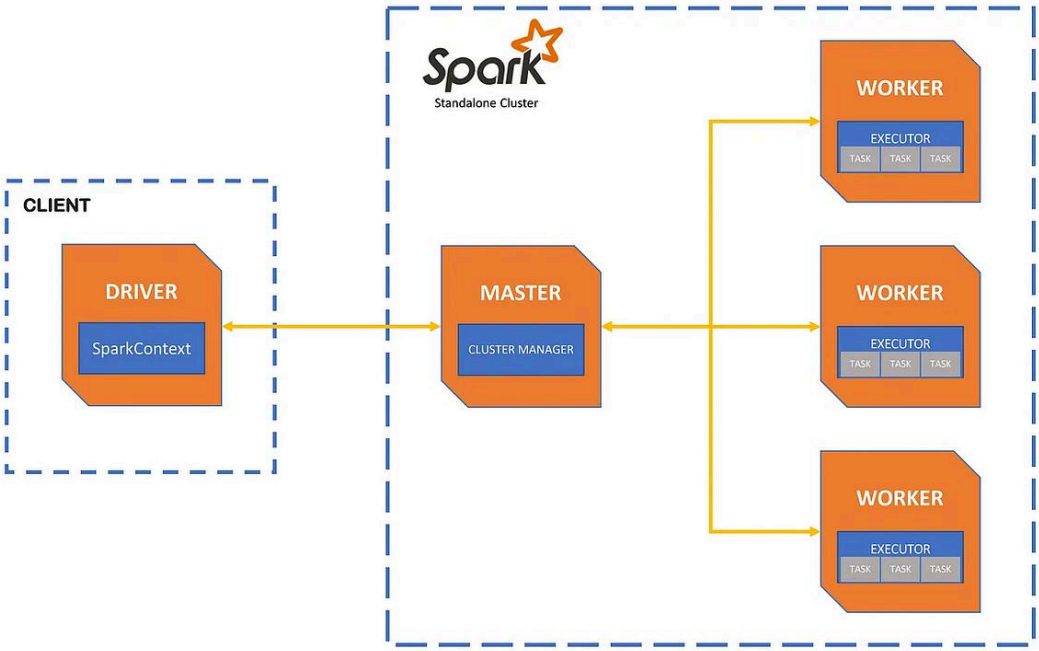
4. Driver requests resources from cluster manager
5. Executors are launched on worker nodes
6. Code is converted into a DAG (Directed Acyclic Graph)
7. DAG is divided into stages and tasks
8. Tasks are distributed to executors
9. Executors process data and return results

This flow ensures efficient parallel execution and fault tolerance, as tasks can be recomputed if failures occur.

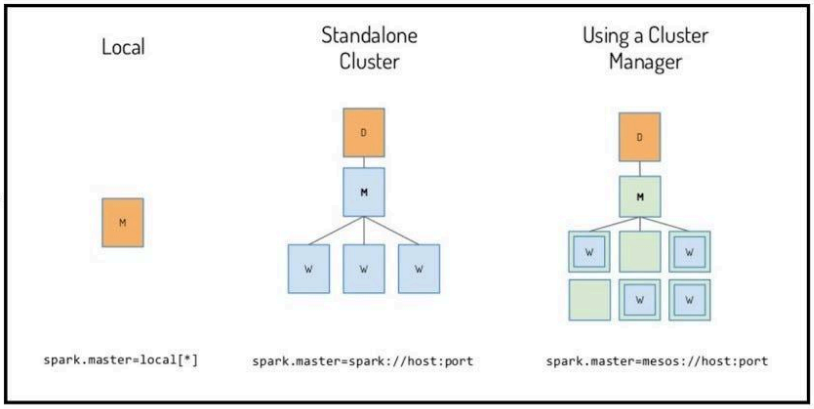
Client Mode vs Cluster Mode

In **Client Mode**, the driver runs on the user's local machine. The application interacts directly with the cluster manager and worker nodes.

This mode is typically used for development and testing because it allows easier debugging. However, it depends on the client machine's availability and network connection.



In **Cluster Mode**, the driver runs inside the cluster, usually on a worker node. The cluster manager is responsible for launching both the driver and executors.

This mode is more suitable for production environments because it provides better fault tolerance and resource management. If the client disconnects, the application continues running inside the cluster.

Deployment Options